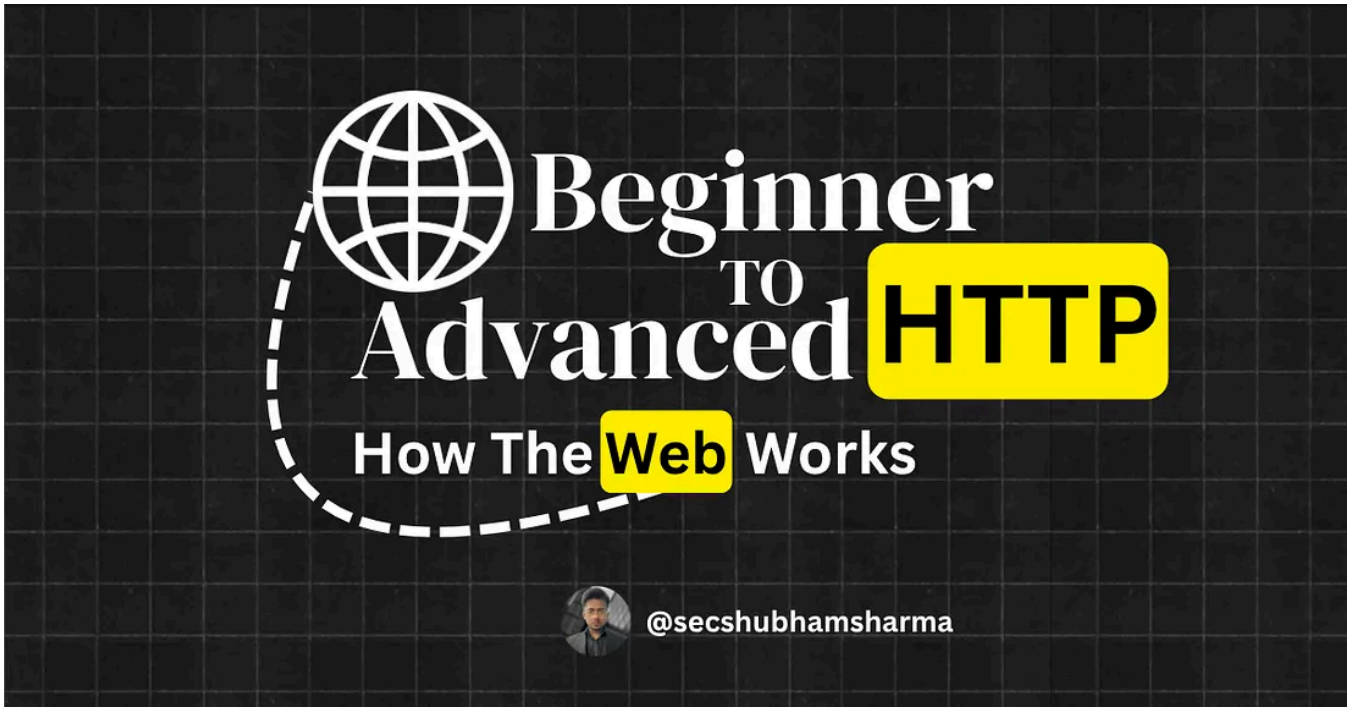
The HTTP Basics to Advanced: Beginner Friendly Guide to How the Web Works

20 min read · Jul 1, 2025

Shubham Sharma Follow

Listen Share



So if you're getting into IT, Networking, Software Development or cybersecurity etc, the first thing you *really* need to learn is how the web communicates. Sit tight and I am going to cover every thing in detail **HTTP basics to Advanced.**

Every time you click a link, submit a form, or open a website you're triggering a conversation. And that conversation happens through HTTP, or its secure version, HTTPS.

Now, don't worry I'm not going to throw a bunch of textbook definitions or complicated networking terms at you. I'm just going to explain what I've learned, in the simplest way possible the way I wish someone had explained it to me when I started.

> *I've also added a handwritten note I personally made, just for you guys to make this journey even easier.*

Alright, let's start.

First Things First: What's a URL?

URL stands for **Uniform Resource Locator**, and it's basically the full address of a resource on the internet. When you type a URL into your browser or call an API endpoint, that string tells the browser exactly where to go and what to fetch.

Now let's break this down into simple parts:

- ***https*** — This is the **protocol**. It tells the browser to use a secure connection.
- ***example.com*** — This is the **domain**. Think of it as the name of the server you're trying to reach.
- ***443*** — This is the **port**. Port 443 is the default for HTTPS, just like port 80 is for HTTP. (In most URLs, the port is hidden because it's default.)
- ***/products*** — This is the **path**. It tells the server what resource or section you want.
- ***?id=5&sort=asc*** — This is the **query string** often called as **Parameter**. It adds extra parameters in this case, you're asking for product number 5, and you want the results sorted in ascending order.

> *A **protocol** is just a **set of rules** that computers follow to communicate with each other.*

Alright, now that we've got the basics of URLs down, let's get into what really happens when you type something like **https://www.example.com** into your browser and hit Enter.

How Your Browser Talks to a Website ?

well now let's say what things goes under the hood, how your browser (client) talks to a website (server).

Step 1: You Type a URL

You enter something like https://www.example.com/index.html. Your browser breaks it down into three parts:

- https — the protocol (a fancy word for the rules of communication)
- www.example.com — the domain name
- /index.html — the page you're asking for

Simple enough, right? But your browser still doesn't know where this site physically lives.

Step 2: DNS Lookup

Now it asks a DNS server, "Hey, where do I find this domain?" Think of DNS like the internet's phonebook. It responds with an IP address, like 93.184.216.34, so your browser knows exactly where to go.

Step 3: TCP Handshake

The browser now knows where to go and tries to connect to the server using something called a TCP connection. It does this using two things:

- The IP address of the server, for example: 93.184.216.34
- The port number, which is usually 80 for HTTP and 443 for HTTPS

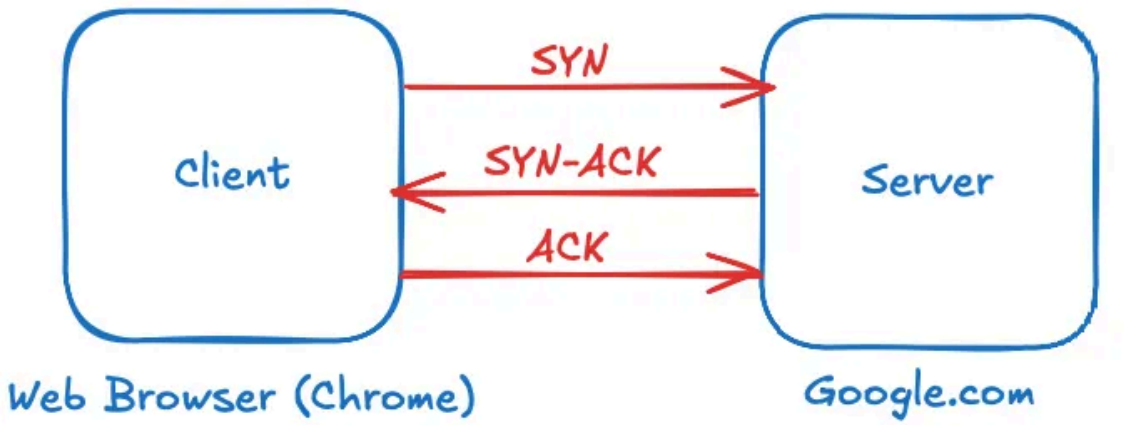
Now, what is TCP?

TCP (Transmission Control Protocol) is a reliable, connection-oriented protocol. That means it makes sure all the data you send and receive arrives in order. If any piece

of data is missing or lost along the way, TCP will request it again to make sure nothing is incomplete.

To start this connection, TCP uses something called a **3-way handshake**, which is a simple back-and-forth conversation to set things up:

1. First, the browser sends a message to the server saying, “Can we talk?” (This is called **SYN**)
2. The server replies, “Yes, let’s talk.” (This is **SYN-ACK**)
3. Finally, the browser responds, “Cool, starting now.” (This is **ACK**)



Once this 3-step handshake is complete, the TCP connection is successfully established and ready for communication.

Step 4: TLS Handshake (Only in HTTPS)

If you’re using https, the browser first checks the server’s identity with a digital certificate. Once verified, they both agree on encryption keys so nobody can spy on their conversation. That’s your secure tunnel right there.

Step 5: The HTTP Request

With everything set, your browser finally says:
“Hey server, give me /index.html, and by the way, I’m Chrome and I’d prefer HTML.”

That's the HTTP request. It includes the method (GET), the path, and a bunch of headers explaining what the browser wants and how it wants it.

```
GET / HTTP/2
Host: www.google.com
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:139.0) Gecko/20100101
Firefox/139.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Upgrade-Insecure-Requests: 1
Referer: https://google.com/

```

Step 6: The Server Responds

The server receives the request, processes it, maybe runs some code or checks a database, and then sends a response back:
"Here's that page you asked for!"

You'll get an HTTP status (like 200 OK), headers describing the content, and the actual HTML body.

```
HTTP/2 200 OK
Date: Mon, 30 Jun 2025 21:48:16 GMT
Expires: -1
Cache-Control: private, max-age=0
Content-Type: text/html; charset=UTF-8
Strict-Transport-Security: max-age=31536000
Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-src
'nonce-88FWbfTeYtUwddl4Mwg-hg' 'strict-dynamic' 'report-sample' 'unsafe-eval'
'unsafe-inline' https: http:;report-uri https://csp.withgoogle.com/csp/gws/other-hp
Cross-Origin-Opener-Policy: same-origin-allow-popups; report-to="gws"
Report-To:
{"group":"gws","max_age":2592000,"endpoints":[{"url":"https://csp.withgoogle.com/csp/r
eport-to/gws/other"}]}
Accept-Ch: Sec-CH-Prefers-Color-Scheme
P3p: CP="This is not a P3P policy! See g.co/p3phelp for more info."
Server: gws
X-Xss-Protection: 0
X-Frame-Options: SAMEORIGIN
Set-Cookie: AEC=AVh_V2hlvxAuyIOausNSMeJAWlRMOdNFXSgFewWUeWVjco6KECe3JW17VA;
expires=Sat, 27-Dec-2025 21:48:16 GMT; path=/; domain=.google.com; Secure; HttpOnly;
SameSite=lax
Set-Cookie: NID=
525=MC2EFtKoQ1qAtEoGBSpDByP0JwdNRYa2tNCcpFoYFIdQK_mbam8iP4KpYsj6-SfpDLBKfTS2x3eu3wSH87
i-frC6Yfn-zFcb3mu6F3E2CJwgiy3wQJUSwRk4e5o2LwyLV56Im0pvkJ9krtYGjyexJGSlECuS2DTY7oemxMnq
DVZ64_ea5qcrK-eFubVWWO6YmzxTe-mMf-VexCFGBw; expires=Tue, 30-Dec-2025 21:48:16 GMT;
path=/; domain=.google.com; Secure; HttpOnly; SameSite=none
Alt-Svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000

<!doctype html><html itemscope="" itemtype="http://schema.org/WebPage" lang="en-IN">
     <head>
          <meta charset="UTF-8">
          <meta content="origin" name="referrer">
          <link href="//www.gstatic.com/images/branding/searchlogo/ico/favicon.ico"
          rel="icon">
          <meta content="/images/branding/googleg/1x/googleg_standard_color_128dp.png"
           itemprop="image">
          <style>
               @font-face{
                    font-family:'Google Sans';
                    font-style:normal;
                    font-weight:400700;
                    font-display:optional;
                    src:url(//fonts.gstatic.com/s/googlesans/v29/4UaGrENHsxJlGDuGo1
                    OIlL3Owp5eKQtG.woff2)format('woff2');
                    unicode-range:U+0000-00FF,U+0131,U+0152-0153,U+02BB-02BC,U+02C6,U+
                    02DA,U+02DC,U+0304,U+0308,U+0329,U+2000-206F,U+20AC,U+2122,U+2191,
                    U+2193,U+2212,U+2215,U+FEFF,U+FFFD;
               }
          </style>
```

Step 7: The Browser Builds the Page

Now the browser takes over. It starts parsing the HTML and goes:
“Oh look, there’s a CSS file, let me fetch that.”
“Ah, JavaScript? Gotta grab that too.”
“Images? Don’t worry, I’m on it.”

It sends out more HTTP requests for everything linked in that page and assembles it all like a puzzle. Within milliseconds, the full website appears on your screen.

What is HTTP?

HTTP stands for **HyperText Transfer Protocol**. Yeah, it sounds technical, but at its core, it's simply the method your browser and a web server use to talk to each other. **The port for HTTP is 80 by default**

Whenever you type in a website address or click on something, your browser sends a request to the server asking for content. That content could be an HTML page, a file, an image, or even data from an API. The server replies with the requested content, and your browser displays it.

Remember **HTTP is Stateless** means the server doesn't remember anything about your previous requests. If you refresh the same page five times, the server treats each one like a brand-new request from a total stranger. No memory, no context, no history.

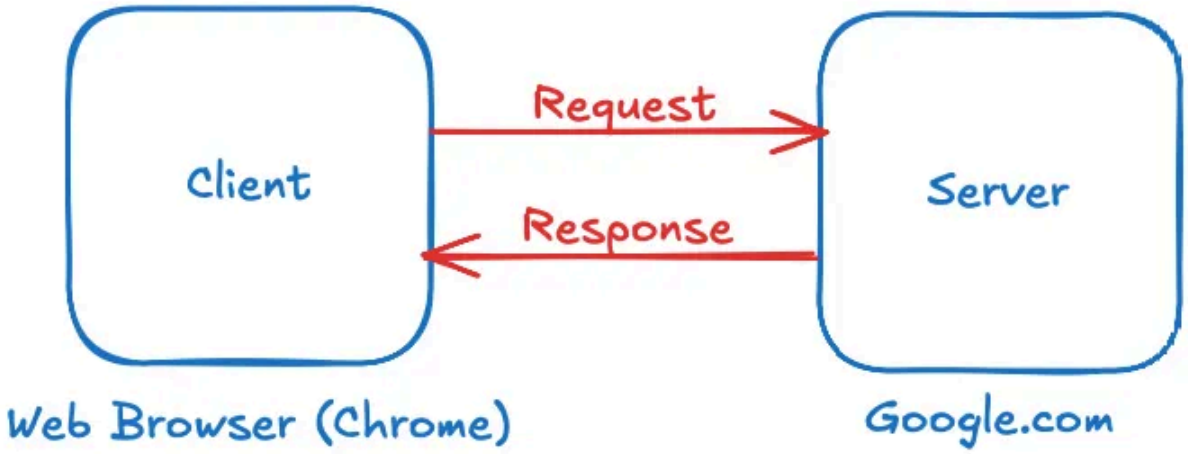
Ok now you must be wondering man what rubbish the website logs us In and we are able to maintain our session how can you say this. Well that's totally fair questions, The truth is That's where **cookies**, **sessions**, and **tokens** come in. These are extra mechanisms layered on top of HTTP to simulate memory. They help websites track who you are across multiple requests even though the protocol itself doesn't remember anything

So one more thing to remember which is HTTP is based on client-server model and you must be wondering now what is the client server model

Understanding The Client-Server Model

The **client** is the one making the request. That could be your browser, a mobile app, or even tools like Postman or curl. It's basically the thing that says, "Hey, I need something."

The **server**, on the other hand, is the one that listens, processes that request, and sends back a response. It's where the website or app actually lives.



What is HTTPS and Why do we need HTTPs?

HTTPS stands for **HyperText Transfer Protocol Secure**. It's basically the secure version of HTTP the way your browser and a website talk to each other. The problem with HTTPs is it sends everything in plain text. The **port for HTTPs is 443 by default**

That means if you're on public Wi-Fi or a compromised network, anyone sniffing the traffic can literally see everything you send your login credentials, personal info, maybe even your sessions.

HTTPs encrypts everything between your browser and the server using **TLS (Transport Layer Security)**, It also makes sure you're actually talking to the real server, not some sketchy middleman trying to fake it.

How HTTPs actually works behind the scene (Not Going In Detail)

Before any real data is exchanged, the browser and server go through a quick process called a **TLS handshake**.

Steps:

- Server sends its **digital certificate**
- Browser verifies it's valid and trusted
- They agree on **encryption keys**
- A **secure encrypted connection** is established

well now let's learn about the HTTP methods

HTTP Methods

Whenever your browser or an app sends a request to a server, it includes a method that tells the server what it wants to do. Are we asking for something? Sending something? Updating? Deleting? These methods define the purpose of the request.

1. GET Method

The **GET** method is used when the client wants to *retrieve* information from the server. It doesn't change anything on the server it simply asks for data.

GET requests are **safe**, **read-only**, and **idempotent**, meaning no matter how many times you make the request, the result stays the same (unless the data changes on its own).

Example:
Imagine you visit google.com. Your browser sends a GET request to the server asking for the content of that page.

```
GET / HTTP/2
Host: www.google.com
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:139.0) Gecko/20100101
Firefox/139.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Upgrade-Insecure-Requests: 1
Referer: https://google.com/

```

The server replies with the google search home page.

2. POST — Submitting Data or Creating a Resource

The **POST** method is used when you want to *send data to the server* usually to create something new or trigger a process.

Unlike GET, POST requests can **change server state**, and they're **not idempotent** — repeating the same request could create multiple records or actions.

Example:
When a user signs up on a website, the form submission is handled with a POST request or suppose say you are uploading your photo on facebook

```
POST /register HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "username": "john_doe",
  "password": "securePassword123"
}
```

The server creates a new user and responds with a confirmation or in facebook your photo will be uploaded

3. PUT — Updating or Replacing a Resource

The **PUT** method is used to completely *update or replace* an existing resource. It's like saying, "Here is the new version of this item."

PUT requests are **idempotent** sending the same data multiple times results in the same final state.

So before you are going to get confused with PUT and PATCH, Just remember on thing when you are using PUT then all the resources get updated.

Example:
Let's say a user wants to update their profile information.

```
PUT /user/123 HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "name": "John Doe",
  "email": "john@example.com"
}
```

The server replaces the current user data with this new one.

4. PATCH — Making a Partial Update

The **PATCH** method is similar to PUT but more efficient when you only want to *update specific fields* instead of replacing the entire resource.

It's useful when only a small change is needed.

Example:
If a user just wants to update their email address, a PATCH request would look like:

```
PATCH /user/123 HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "email": "newemail@example.com"
}
```

5. DELETE — Removing a Resource

The **DELETE** method is used to *remove* a resource from the server.

Like PUT and PATCH, DELETE is **idempotent** deleting something that doesn't exist still returns a valid response.

Example:
A user decides to delete their account:

```
DELETE /user/123 HTTP/1.1
Host: example.com
Authorization: Bearer <auth_bearer>
Content-Type: application/json
Content-Length: 77

```

The server deletes the user with ID 123 and returns a confirmation.

6. HEAD — Checking Without Downloading

The **HEAD** method is almost identical to GET, except it only returns the **headers**, not the body (content). It's useful when you want to check if a resource exists or inspect its metadata.

Example:
You want to know if a file is available before downloading it:

```
HEAD /files/report.pdf HTTP/1.1
Host: example.com
Authorization: Bearer <auth_bearer>
Content-Type: application/json
Content-Length: 77
```

The server replies with the headers like `Content-Length`, `Content-Type`, and `User-Agent`. well don't panic i will discuss about this as well.

7. OPTIONS — Discovering What's Allowed

The **OPTIONS** method asks the server what it supports for a specific resource or endpoint. It's often used for **CORS (Cross-Origin Resource Sharing)** preflight checks in web browsers.

Example:
Your browser checks what methods are allowed before sending a POST request:

```
OPTIONS /api/user HTTP/1.1
Host: example.com
Authorization: Bearer <auth_bearer>
Content-Type: application/json
Content-Length: 77
```

So, the server will reply you with allowed HTTP Method that Endpoint (URI) Accepts.

for an example like this in the response in the Allow header

```
Allow: GET, POST, PATCH, DELETE
```

8. CONNECT — Creating a Tunnel for Secure Communication

The **CONNECT** method is used to *create a tunnel* between the client and server usually for **HTTPS** traffic through an HTTP proxy.

It's not something you use in your own requests but is essential for secure browsing behind the scenes.

Example:
A browser connecting to an HTTPS website via a proxy server:

```
CONNECT www.secure-site.com:443 HTTP/1.1
Host: ecure-site.com
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:139.0) Gecko/20100101
Firefox/139.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: https://iplocation.com/
Sec-Fetch-Dest: script
Sec-Fetch-Mode: no-cors
Sec-Fetch-Site: cross-site
Te: trailers
Connection: keep-alive
```

The proxy sets up a tunnel so the browser can talk securely to the site.

9. TRACE — Debugging the Request Path

The **TRACE** method is used for *diagnostics and debugging*. It asks the server to return the exact request it received, so the client can see if anything was altered along the way.

It's rarely used today and often **disabled** for security reasons (to prevent Cross-Site Tracing attacks).

Example:

A developer want to diagnose or debug the server

```
TRACE /debug HTTP/1.1 HTTP/1.1
Host: secure-site.com
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:139.0) Gecko/20100101
Firefox/139.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: https://iplocation.com/
Sec-Fetch-Dest: script
Sec-Fetch-Mode: no-cors
Sec-Fetch-Site: cross-site
Te: trailers
Connection: keep-alive

```

The server echoes the full request it received.

The Difference Between PUT and PATCH

Let's first understand what PUT does.

Imagine you're on Facebook and you want to update your name. Now, hypothetically, if Facebook used the **PUT** method to handle that update, here's what would happen:

When you submit the request, **Facebook would need to send all your details** your name, email, address, and any other profile info*even if you only changed your name*.

Why? Because **PUT replaces the entire resource**. If you leave out any field, it could be considered as `null` or reset to a default value. So to avoid accidentally wiping out your email or address, the full data has to be included in the request.

```
PUT /api/user HTTP/1.1
Host: example.com
Authorization: Bearer <auth_bearer>
Content-Type: application/json
Content-Length: 77

{
  "name": "Shubham Sharma",
  "email": "shubham@example.com",
  "address": "Delhi, India"
}
```

What Patch Does ?

Let's say you only want to update your name again. With PATCH, you **only send the name field** in the request. That's it.

The server will update just that one piece and leave everything else your email, address, and other data untouched.

So in short:

- **PUT** = “Here’s the full updated version of everything.”
- **PATCH** = “Just change this specific part, leave the rest as it is.”

```
PATCH /api/user HTTP/1.1
Host: example.com
Authorization: Bearer <auth_bearer>
Content-Type: application/json
Content-Length: 77

{
  "name": "Shubham Sharma"
}
```

Once the client (like your browser) sends a request, the server replies with a **response** and with it comes a **status code,** now let us take a look into it.

What Are Status Codes?

This code quickly tells you what happened: whether the request was successful, failed, redirected, or blocked.Let’s go through the most common ones and what they actually mean.

It’s the server’s quick way of saying, “Your request worked,” or “Something went wrong,” or even “Try again somewhere else.”

well now let us look into the various status code we have, well i would be only discussing the more used and usually you will see.

🟦 1xx — Informational Responses

They indicate that the request was received and the process is continuing. These are mostly used under the hood during protocol upgrades or ongoing data transfers. You don't usually see them in everyday browsing or development unless you're dealing with advanced networking or streaming protocols.

These codes mean *"Hold on, I'm working on it."*

1. 101 Switching Protocols

Definition:
The client has asked the server to switch protocols (like from HTTP to WebSocket), and the server agrees.

Use Case:
A chat app requests to upgrade the connection to WebSocket after an initial HTTP handshake. The server responds with 101 to confirm the switch.

✅ 2xx — Success

The server successfully received, understood, and accepted the request. These are the best responses you want to see as a developer or tester.

This category means *"Everything went fine."*

1. 200 OK

Definition:
The request was successfully received, understood, and processed by the server.

Use Case:
A GET request to `/products` returns the list of items in JSON format with a 200 OK response.

2. 201 Created

Definition:
The request was successful and led to the creation of a new resource.

Use Case:
Sending a POST request to `/users` with registration data returns 201 when a new user is created in the database.

3. 204 No Content

Definition:
The request was processed successfully, but there's no content to return in the response body.

Use Case:
After successfully deleting a record using DELETE `/user/45`, the server returns 204 with no response body.

🔁 3xx — Redirection

The request was received, but to complete it, the client needs to make another request to a different URL. This is often used in login flows, content migrations, or domain redirections.

These codes mean *"You're not quite there yet go somewhere else."*

1. 301 Moved Permanently

Definition:
The requested resource has permanently moved to a new URL.

Use Case:
An old blog URL `/blog/welcome` is permanently redirected to `/articles/welcome`. The browser is automatically redirected with a 301 status.

2. 302 Found

Definition:
The resource is temporarily located at another URI, but the client should keep using the original URI for future requests.

Use Case:
After logging in through `/login`, the user is temporarily redirected to `/dashboard` using a 302 status.

3. 303 See Other

Definition:
After a POST request, the server redirects the client to a different URL to retrieve the result using GET.

Use Case:
Submitting a form on `/payment` returns 303 with the `Location` header pointing to `/payment/summary`.

⚠️ 4xx — Client Errors

The request had an error on the client's side wrong syntax, missing info, or unauthorized access. These codes help developers debug what went wrong with a request.

These indicate *"You did something wrong."*

1. 400 Bad Request

Definition:
The server couldn't understand the request due to invalid syntax, missing fields, or malformed data.

Use Case:
Sending an API request to `/api/user` without a required `name` field in JSON payload results in a 400 response.

2. 401 Unauthorized

Definition:
Authentication is required and was either not provided or failed.

Use Case:
Trying to access `/user/profile` without a valid JWT token returns a 401 Unauthorized response.

3. 403 Forbidden

Definition:
The client is authenticated but doesn't have permission to access the requested resource.

Use Case:
A logged-in user without admin rights attempts to access `/admin/panel` and gets a 403 response.

4. 404 Not Found

Definition:
The server couldn't find the requested resource.

Use Case:
Visiting `/api/posts/9999` returns 404 if the post with ID 9999 doesn't exist in the database.

5. 405 Method Not Allowed

Definition:
The HTTP method used is not supported for the requested resource.

Use Case:
Trying to send a DELETE request to `/contact-form` which only supports GET and POST results in a 405.

6. 409 Conflict

Definition:
The request could not be completed due to a conflict with the current state of the resource.

Use Case:
Attempting to create a user with an email that already exists in the system returns a 409 Conflict.

7. 429 Too Many Requests

Definition:
The user has sent too many requests in a short period of time.

Use Case:
Sending 100 login attempts within a minute triggers rate limiting and returns a 429 status.

🔴 5xx — Server Errors

You sent a valid request, but the server couldn't handle it due to an internal issue. These are typically bugs, crashes, or misconfigurations in backend services or APIs.

This group means *"The server messed up."*

500 Internal Server Error

Definition:
The server encountered an unexpected condition that prevented it from fulfilling the request.

Use Case:
A database query throws a null pointer exception while processing a POST `/submit-form`, returning a 500 error.

1. 501 Not Implemented

Definition:
The server does not support the functionality required to fulfill the request.

Use Case:
Making a PATCH request to a legacy server that doesn't support it returns a 501 Not Implemented.

2. 502 Bad Gateway

Definition:
The server was acting as a gateway or proxy and received an invalid response from the upstream server.

Use Case:
An Nginx server forwards a request to a backend API, but the backend crashes. Nginx returns a 502 to the client.

3. 503 Service Unavailable

Definition:
The server is currently unable to handle the request, often due to being overloaded or under maintenance.

Use Case:
During scheduled server maintenance, requests to `/checkout` return a 503 with a "Retry-After" header.

4. 504 Gateway Timeout

Definition:
The server acting as a gateway or proxy did not receive a timely response from the upstream server.

Use Case:
A frontend server calls a slow analytics microservice. When it doesn't respond in time, the server returns a 504 error.

These are the main status codes you'll come across while working with the web.

Ok, now i hope at this point you have all good understanding of HTTP, HTTP Method and HTTP Status Code, Now let's try to read some HTTP request and understand it

Understanding How To Write HTTP Request

In this section, I'm going to dig into what a real HTTP request actually looks like, and how you can write one yourself. If you're trying to understand how this works in practice, I really recommend trying it out hands-on. Writing a few HTTP requests manually will help everything click.

You can use tools like **Postman** (great for beginners) or **Burp Suite** if you're diving into security testing. Both let you build and send custom requests easily. I won't be covering how to use these tools here that would take us a bit off track. Maybe I'll save that for another article.

So the First Line HTTP Request starts with

At first we have to write the HTTP Method so in this case just trying to load a web page in this case, we use the `GET` method. Now after a space we have to give the Path we want to access, And after a space we have to write the HTTP Version. This is how the first line of HTTP Request looks like

```
GET /about HTTP/1.1
```

And now let's say you want to create a post on Facebook — instead of just viewing something, you're sending data to the server. In that case, the HTTP method changes from `GET` to `POST`.

So, the first line of your HTTP request would look like this

```
POST /create-post HTTP/1.1
```

Just like before, we start with the method, then a space, followed by the path we're trying to access, and finally, the HTTP version.

And remember **HTTP methods are always written in uppercase.**

Now since we have understood the first line, We won't be searching for the inner.html out of thin air right there must be a domain name (website) in which we would be requesting the inner.html, so in the second line we have to give the host headers, means the server from which we want the request

well that the first 2 things required, other headers that might be included in the request. These headers give the server more details about what the client is sending or what it expects in return.

NOTE: Not all headers are required. Some only show up depending on what kind of request you're making, what the server supports, or how the browser decides to communicate. It really varies depending on the situation.

So to understand this let's look into a example request

```
POST /v1/users?client_version=11.9.0&client_id=793f4dd7a71b&features=
login-change,static-drafts,drafts-versioning,style-parts,modified-calendars,unseen-assignment,xfer-accel,yea
rly-plans,device-authentication,external-templates,new-tasks,oemt&tz=Asia/Kolkata&co=2&_method=GET HTTP/2
Host: api.example.com
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:139.0) Gecko/20100101 Firefox/139.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: text/plain
Content-Length: 367
Referer: https://mail.example.com/
Origin: https://mail.example.com
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-site
Priority: u=4
Te: trailers

{
     "auth_token":"here will be an auth token ",
     "_debug":{
          "bootstrapped_at":1751286471,
          "last_server_broadcast":1751312247,
          "url":"https://mail.example.com.com/#search/2fa/conversations/a135bce120e7"
     }
}
```

Host

This one's simple. It tells the server which website you're trying to talk to super useful when the same server is hosting multiple sites.

Example:

```
Host: api.example.com
```

User-Agent

This tells the server what kind of browser and operating system you're using. So if you're on a Mac with Firefox, the server knows that and might adjust things to work best for you.

Example:

```
User-Agent: Mozilla/5.0 (Macintosh...) Firefox/139.0
```

Accept

Here, your browser is saying what kind of content it's okay with. If it's set to `*/*`, that just means "I'm cool with anything HTML, JSON, whatever."

Example:

```
Accept: */*
```

Accept-Language

This one tells the server your preferred language. So if you're using English, it'll try to send you stuff in that language if it can.

Example:

```
Accept-Language: en-US,en;q=0.5
```

Accept-Encoding

This lets the server know what types of compression your browser can handle. Things like `gzip` and `br` help speed up loading times because they make files smaller.

Example:

```
Accept-Encoding: gzip, deflate, br
```

Content-Type

This one's for when you're sending data like submitting a form or uploading something. It tells the server what kind of data you're sending, so it knows how to read it.

Example:

```
Content-Type: text/plain
```

Content-Length

This just says how big the body of the request is in bytes. The server uses this to figure out when it's received the whole thing.

Example:

```
Content-Length: 367
```

Referer

This shows where the request came from like which page you were on when you clicked a button or submitted a form.

Example:

```
Referer: https://mail.example.com/
```

> By the way, yes, "Referer" is a typo. It was supposed to be "Referrer" but they messed up in the original spec, and now we're all stuck with it.

Origin

Kind of like Referer, but it just includes the domain no path or extra details. Mostly used in security stuff like CORS to check if the request is coming from a trusted site.

Example:

```
Origin: https://mail.example.com
```

Sec-Fetch-Dest

This tells the server what kind of thing you're trying to get like an image, a script, or something else. If it says `empty`, it just means there's no specific "visual" thing being fetched.

Example:

```
Sec-Fetch-Dest: empty
```

Sec-Fetch-Mode

This one explains how the request is being made. If it says `cors`, it means it's a cross-origin request (like when you're trying to access a different domain's server). Well cors is a big part will try to cover the full cors in a different article.

Example:

```
Sec-Fetch-Mode: cors
```

Sec-Fetch-Site

This tells the server if the request is coming from the same site or a different one. It helps with security, especially against CSRF attacks.

Example:

```
Sec-Fetch-Site: same-site
```

Priority

Not every server uses this, but it's a little hint that tells the browser how important this request is compared to others. Lower numbers usually mean higher priority.

Example:

```
Priority: u=4
```

Te

This one's not super common, but it basically says the browser is okay with receiving some extra headers at the *end* of the response used in chunked data situations.

Example:

```
Te: trailers
```

BODY

Alright, finally we have the body this is where the actual data of the request goes. What it looks like totally depends on what kind of request you're sending.

If you're sending HTML form data, it'll be formatted like a form.
If you're sending JSON, it'll be structured like a JSON object.

Just one important thing to remember:
There **must be a blank line between the headers and the body** that's how the server knows the headers have ended and the body is starting. If you Miss that then the request might not even go through properly.

Once you understand these HTTP headers, everything starts to make more sense when you're looking at requests whether you're building stuff, testing APIs, or just learning how the web works. They might look technical at first, but they're really just your browser and the server chatting with some extra details.

And that brings us to the end of this article.
Hope it added some value to your learning, and maybe helped you understand a few things a little clearer

Conclusion

Once you understand these HTTP headers, everything starts to make more sense when you're looking at requests whether you're building stuff, testing APIs, or just learning how the web works. They might look technical at first, but they're really just your browser and the server chatting with some extra details.

Keep playing around with real requests using tools like Postman, stay curious that's how you really learn this stuff.

In case you have any questions or anything you can connect with me

Linkedin: https://www.linkedin.com/in/secshubhamsharma/

Thank You, For reading till the end

Https | Networking | Bug Bounty | Web Development | Software Development

Follow

Written by Shubham Sharma

21 followers · 3 following

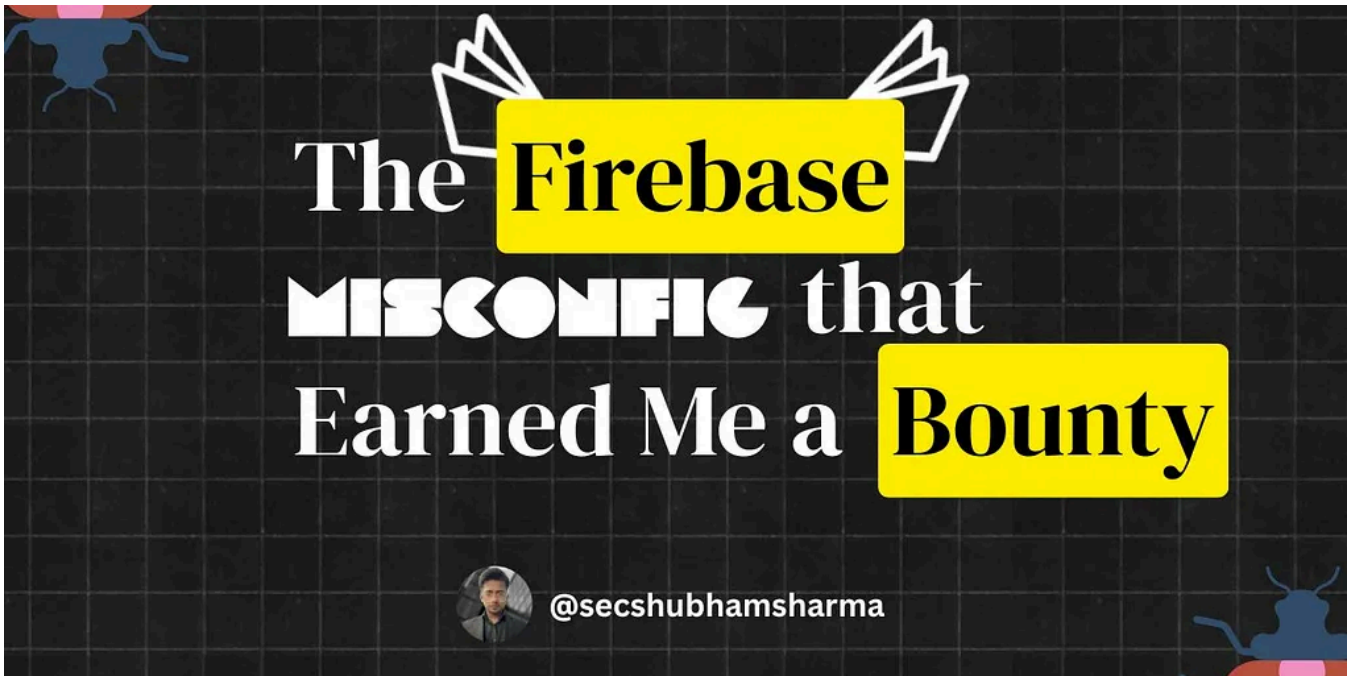
Security researcher | Reported 1200+ bugs on HackerOne & Bugcrowd | Cloud & app security | Connect → linkedin.com/in/secshubhamsharma

No responses yet

Write a response

What are your thoughts?

More from Shubham Sharma

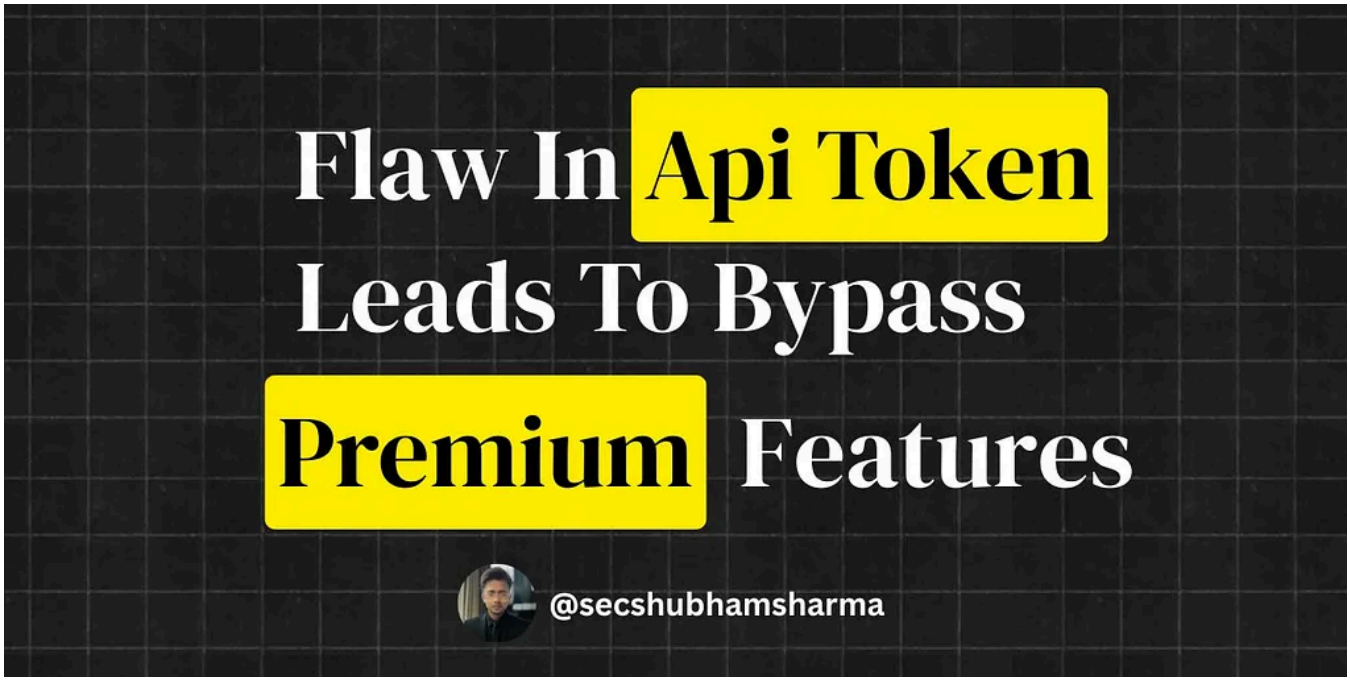


Shubham Sharma

Bug Hunting 101: The Firebase Misconfig That Earned Me a Bounty

Hey folks, welcome to another article, well this is my first bounty writeup. Today, I'm going to share something really interesting, where...

Jul 12 12 1

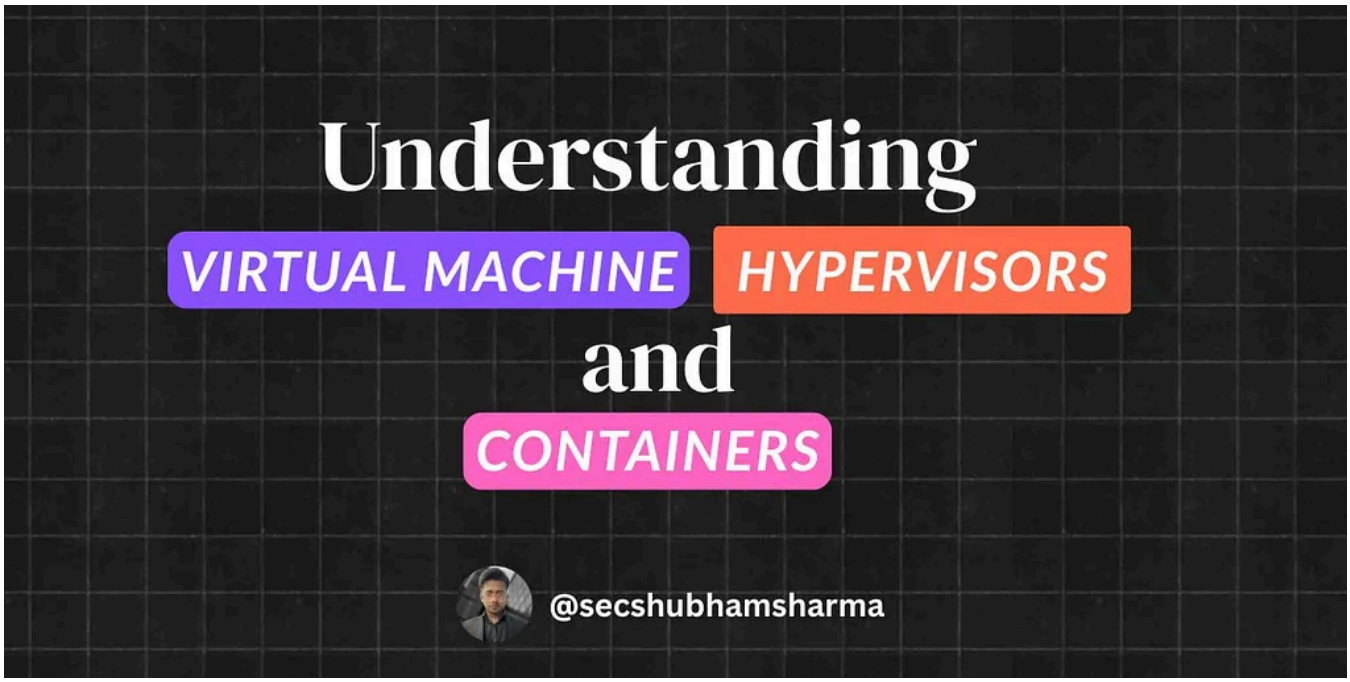


In System Weakness by Shubham Sharma

How an API Token Flaw Let Me Bypass Premium Restrictions

Hi Guys, Welcome back to yet another article, Today, I want to walk you through one of the bugs I found while hunting on HackerOne. I was...

Aug 5 11

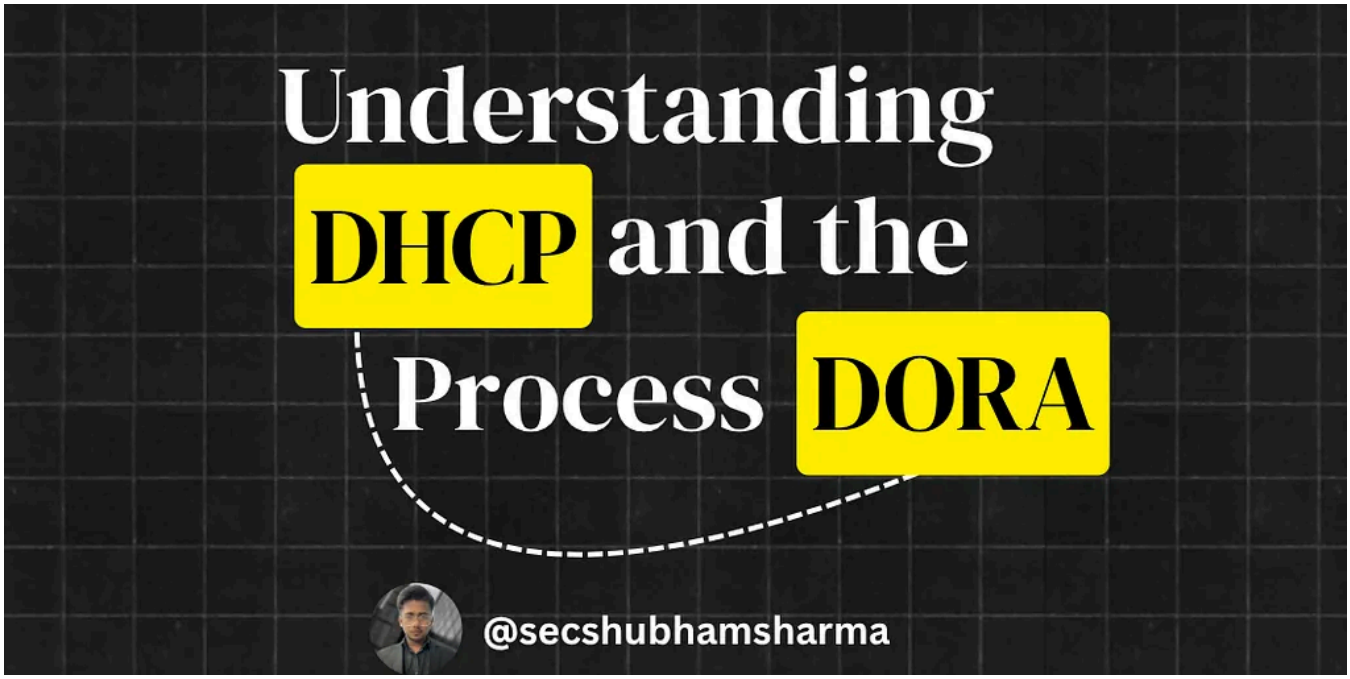


Shubham Sharma

Understanding Virtualization, Hypervisors and Containers.

Hey folks, welcome to another article! Today, I am going to break down something which is backbone of modern infrastructure...

Jul 16



Shubham Sharma

A Beginner’s Guide to Understanding DHCP and the DORA Process

Hey folks, welcome to another article. In this article, I’m going to talk about DHCP and the DORA process, covering everything about it.

Jul 9

See all from Shubham Sharma

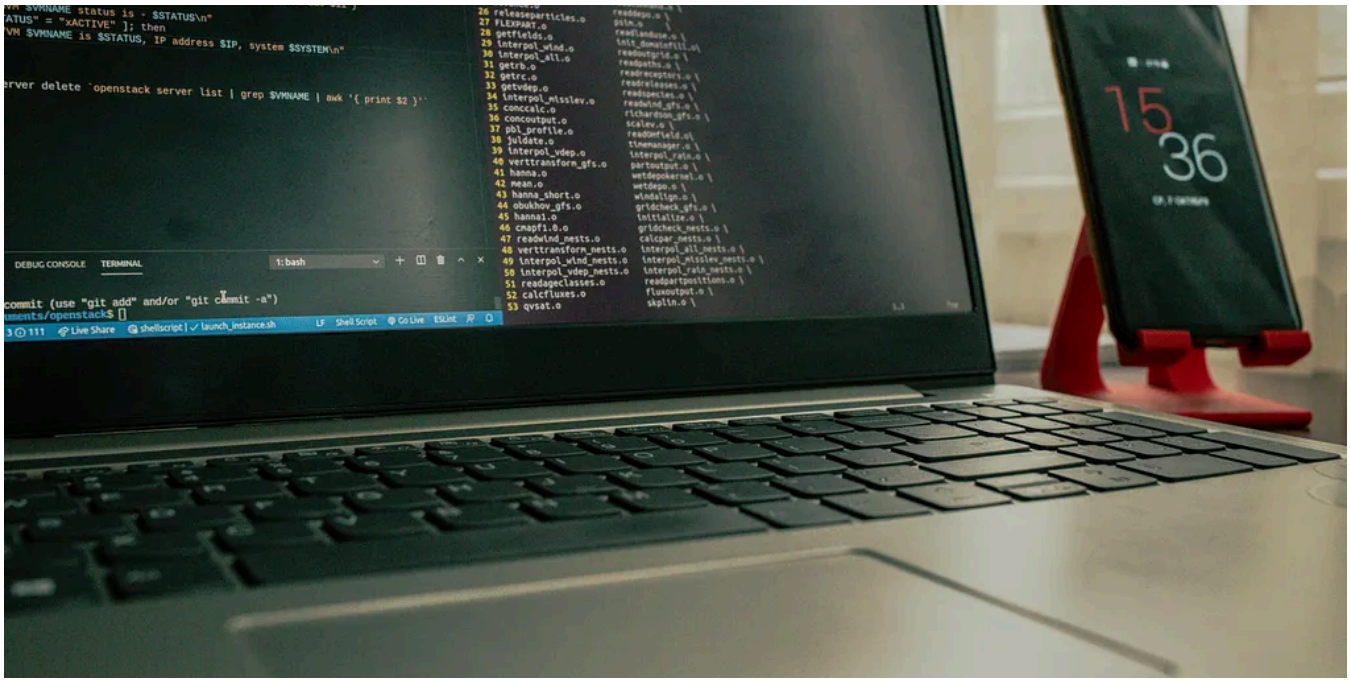
Recommended from Medium

In InfoSec Write-ups by coffinxp

My 5-Minute Workflow to Find Bugs on Any Website

A step-by-step guide to my most effective, shortcut methods for bug bounty hunting.

Sep 27 846 8



Very Lazy Tech

The Ultimate Hacker’s Bash Cheat Sheet (20+ Advanced One-Liners Inside)

✨ Link for the full article in the first comment

Sep 10 230 2

Karim Hikal

Bypass Password Confirmation on Change Email

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ “يَا أَيُّهَا الَّذِينَ آمَنُوا كُلُوا مِن طَيِّبَاتِ مَا رَزَقْنَاكُمْ وَاشْكُرُوا لِلَّهِ إِن...

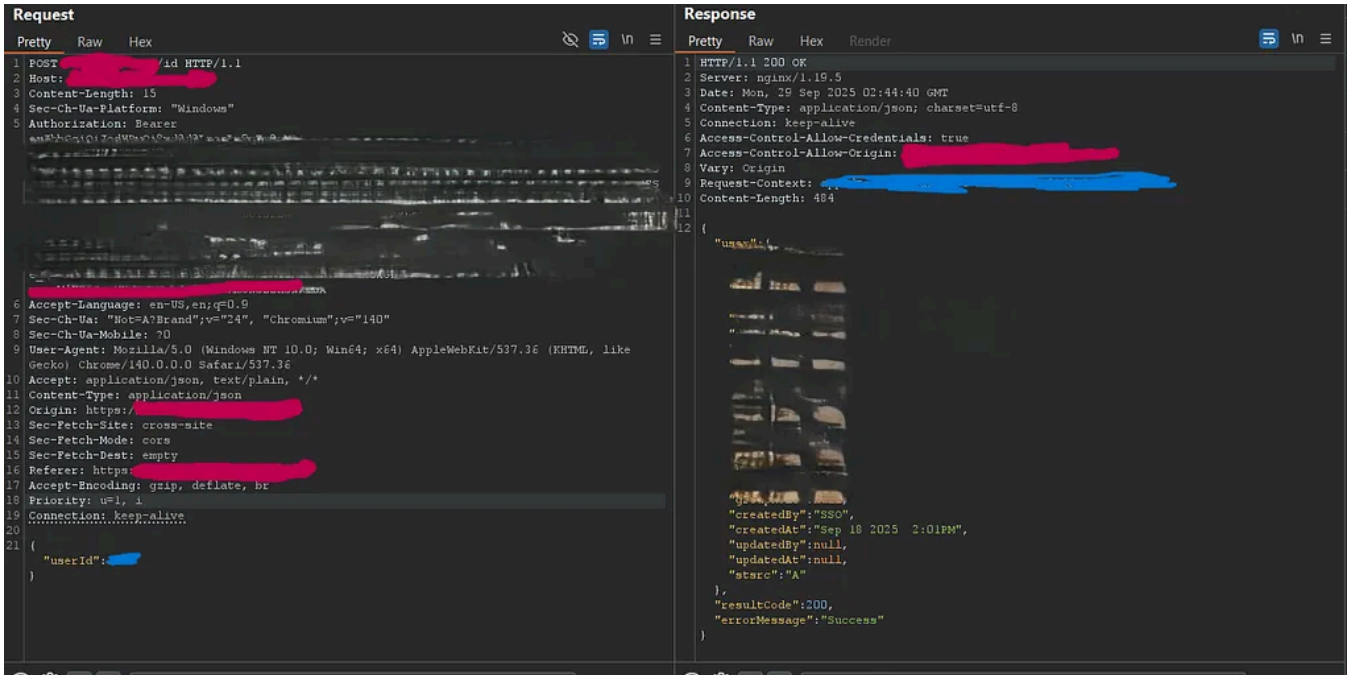
Sep 30 378 5

In Flutter Minds by Sumit Kumar

🔓 Part 1: Offline Login Bypass in Flutter — How SharedPreferences Can Let Attackers Skip...

Welcome to Part 1 of the Flutter App Hacking & Security series. In this article, we'll dissect a critical vulnerability that allows...

Aug 3 1 1



Irsyad Muhammad Fawwaz

Authentication bypass via sequential user IDs in Microsoft SSO integration | Critical Vulnerability

If you’re a penetration tester or bug bounty hunter, never skip SSO in your tests. It’s one of those features that everyone assumes is...

Sep 29 200 5

In Stackademic by Oliver Foster

The 10 Commandments of High-Quality Logging

My article is open to everyone; non-member readers can click this link to read the full text.

4d ago 106

See more recommendations